

6-Week Learning & Project Development Report

Python Backend Engineering, RAG Systems & AI Agents

Introduction

Over the past 6 weeks, I completed a structured learning and implementation program focused on backend engineering, software architecture, testing, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and AI Agents.

The training was designed to progressively build skills from Python fundamentals to advanced AI-powered systems. Each week focused on a specific area of backend and AI development, with hands-on projects implemented using modern industry tools and practices.

The technologies used throughout the program included:

- Python
- FastAPI
- PostgreSQL
- SQLAlchemy
- Pytest
- Pydantic
- LangChain
- OpenAI APIs
- PGVector
- Git & GitHub

The overall objective was to develop production-ready applications while following clean coding principles, testing methodologies, scalable architecture patterns, and modern AI integration techniques.

Week 1 — Python, OOP & Code Quality

Objective

Build strong Python fundamentals and clean coding habits.

Topics Covered

Environment Setup & Git

- Installed and configured Python environments
- Created and managed virtual environments using `venv`
- Configured VS Code for Python development
- Learned Linux command-line operations
- Set up Git and GitHub repositories
- Practiced Git workflows:
 - clone
 - add
 - commit
 - push
 - pull
 - branch management

Core Python

Learned fundamental programming concepts:

- Variables and data types
- Lists, tuples, dictionaries, and sets
- Conditional statements
- Loops and iterations

Functions & File Handling

Implemented:

- Reusable functions
- Function arguments and scope
- File handling using text and CSV files
- Exception handling using try-except blocks

Code Quality & Validation

Improved code quality by using:

- Type hints
- Docstrings
- Ruff linter

- Black formatter
- Pydantic validation models

Implemented:

- Single Responsibility Principle (SRP)

OOP & Project Structure

Learned:

- Classes and objects
 - Inheritance
 - Encapsulation
 - Python modules and packages
 - Factory and Singleton design patterns
-

Week 1 Project — Inventory Management Package

Features Implemented

- Read inventory data from CSV files
- Validate data using Pydantic
- Log invalid entries and runtime errors
- Generate low-stock inventory reports
- Structured package-based project architecture

Key Learnings

- Writing maintainable Python code
 - Data validation and safe file processing
 - Importance of clean architecture
 - Organizing scalable Python applications
-

Week 2 — Testing & FastAPI

Objective

Build reliable backend APIs with proper testing practices.

Topics Covered

FastAPI Fundamentals

Implemented:

- REST API concepts
- FastAPI routes and endpoints
- Request and response validation using Pydantic
- Swagger UI documentation

Applied:

- Open/Closed Principle (OCP)

Database & CRUD APIs

Integrated:

- PostgreSQL database
- SQLAlchemy ORM

Implemented:

- CRUD operations
- Database session handling
- Persistent storage architecture

Testing Fundamentals

Learned:

- Testing pyramid concepts
- Test-Driven Development (TDD)
- Red-Green-Refactor workflow

Implemented:

- Unit tests using Pytest

Fixtures & Test Design

Used:

- Pytest fixtures
- Arrange-Act-Assert pattern
- Reusable test data

Mocking & Advanced Testing

Implemented:

- Mocking for files and databases
 - Parameterized testing
 - Error scenario testing
-

Week 2 Project — Inventory FastAPI Service

Features Implemented

- Inventory CRUD APIs
- PostgreSQL integration
- Request validation
- API documentation
- Automated testing

Key Learnings

- Backend API development
 - Database integration
 - Automated testing strategies
 - Writing maintainable test suites
-

Week 3 — Security & API Design

Objective

Build secure and production-ready APIs.

Topics Covered

Authentication (JWT)

Implemented:

- User registration
- Login APIs
- JWT token generation
- Token validation

Authorization (RBAC)

Implemented role-based access:

- Admin
- Manager
- Staff

Restricted APIs using:

- FastAPI dependencies

Data Relationships

Created:

- Category models
- Product-category relationships
- Relational database mappings

Error Handling & Dependencies

Implemented:

- Centralized exception handling
- Consistent API responses
- Dependency injection

SOLID Principles & Design Patterns

Applied:

- Interface Segregation Principle (ISP)
- Liskov Substitution Principle (LSP)
- Dependency Inversion Principle (DIP)

Implemented:

- Decorator design pattern
-

Week 3 Project — Secure Inventory API

Features Implemented

- JWT-secured authentication
- Role-based permissions
- Protected endpoints
- Relational database models
- Full test coverage

Key Learnings

- API security best practices
 - Authentication & authorization flows
 - Dependency-based architecture
 - Production-level API structure
-

Week 4 — LLMs, Prompting & Embeddings

Objective

Understand how Large Language Models work and integrate them into applications.

Topics Covered

LLM Fundamentals

Learned:

- Tokens and token usage
- LLM response generation
- Zero-shot and few-shot prompting
- API cost calculation

Prompt Engineering

Implemented:

- Structured prompts
- Role-based prompting
- Output formatting techniques

OpenAI API Integration

Worked with:

- OpenAI API calls
- Temperature and parameter tuning
- Usage tracking

Embeddings & Vector Databases

Implemented:

- Text embeddings
- PGVector storage
- Similarity search
- Semantic retrieval

LangChain

Used:

- Prompt templates
 - Chains
 - LangChain abstractions
 - Structured LLM workflows
-

Week 4 Project — LLM Query API

Features Implemented

- FastAPI endpoint for LLM queries
- Prompt handling system
- OpenAI API integration
- Usage and cost tracking

Key Learnings

- LLM integration workflows
 - Prompt optimization
 - Embedding-based retrieval concepts
 - AI-powered backend systems
-

Week 5 — RAG Application

Objective

Build a “Chat with Your Data” system using Retrieval-Augmented Generation (RAG).

Topics Covered

Data Ingestion (Multi-Format Support)

Integrated support for:

- PDF files
- CSV files
- DOCX documents
- PPTX presentations

Used LangChain loaders:

- PyPDFLoader
- CSVLoader
- UnstructuredWordDocumentLoader
- UnstructuredPowerPointLoader

Implemented:

- Document normalization
- Chunking strategies

Retrieval Setup

Implemented:

- Embedding generation
- PGVector integration
- Retriever setup
- Similarity search

RAG Pipeline

Built:

- Retrieval-Augmented Generation pipeline
- Context-aware prompt injection
- Grounded response generation

FastAPI Integration

Created:

- `/chat` endpoint
- API-based RAG interaction system

Testing & Improvement

Improved:

- Chunk size tuning
- Chunk overlap optimization
- Prompt refinement
- Response quality evaluation

Week 5 Project — Multi-Format RAG Chat API

Features Implemented

- Multi-format document ingestion
- PGVector embedding storage
- FastAPI chat endpoint
- Context-aware response generation

Key Learnings

- RAG architecture design
- Semantic retrieval systems

- Context injection strategies
 - Building scalable AI pipelines
-

Week 6 — Building AI Agents

Objective

Build intelligent AI agents capable of reasoning, planning, and tool usage.

Topics Covered

Introduction to Agents

Learned:

- Difference between:
 - Prompt-response systems
 - RAG systems
 - AI agents

Understood:

- Tools
- Planning
- ReAct reasoning pattern

Tools & Function Calling

Implemented tools for:

- Calculator operations
- Database queries
- API calls

Used:

- LangChain tools
- Function calling workflows

Building a Simple Agent

Built an agent capable of:

- Receiving user queries
- Selecting tools dynamically
- Executing actions
- Returning final responses

Agent + RAG Integration

Integrated:

- RAG as an agent tool
- Multi-tool workflows
- Search + calculator + database tools

FastAPI Integration & Control

Created:

- `/agent` endpoint

Implemented safeguards:

- Tool usage limits
 - Loop prevention
 - Logging and monitoring
-

Week 6 Project — AI Agent Service

Features Implemented

- FastAPI-based AI agent service
- Multi-tool support
- RAG integration
- Multi-step reasoning workflows
- Logging system for agent reasoning

Key Learnings

- AI agent architecture
- Tool orchestration

- ReAct reasoning
 - Hybrid AI systems
 - Agent safety and control mechanisms
-

Challenges Faced

Throughout the program, several technical challenges were encountered:

- Managing package dependencies and environment setup
- Debugging FastAPI request/response validation issues
- Handling PostgreSQL connection and migration issues
- Designing scalable project structures
- Improving RAG retrieval accuracy
- Handling chunking and embedding optimization
- Integrating multiple tools inside agents
- Preventing infinite reasoning loops in agents
- Writing reliable automated tests

These challenges helped improve debugging, architecture design, and problem-solving skills.

Overall Skills Gained

Backend Development

- FastAPI
- REST APIs
- PostgreSQL
- SQLAlchemy
- Authentication & Authorization

Software Engineering

- Clean code principles
- SOLID principles
- Design patterns
- Testing methodologies

- Project structuring

AI & LLM Engineering

- OpenAI API integration
- Prompt engineering
- Embeddings
- Vector databases
- RAG pipelines
- LangChain workflows
- AI agents

Dev Tools & Workflow

- Git & GitHub
 - Virtual environments
 - Ruff & Black
 - Pytest
 - Logging and debugging
-

Final Outcome

By the end of the 6-week program, I successfully developed:

- Structured Python applications
- Secure FastAPI backend services
- Fully tested APIs
- LLM-powered applications
- Retrieval-Augmented Generation systems
- AI agents capable of reasoning and tool usage

The program significantly improved both software engineering fundamentals and practical AI application development skills.

It provided hands-on experience in building scalable, maintainable, and production-oriented systems combining backend engineering with modern AI technologies.

Conclusion

This 6-week learning journey provided a strong foundation in backend engineering, clean architecture, testing practices, and AI application development.

Starting from Python fundamentals and progressing toward intelligent AI agents allowed me to understand the complete lifecycle of modern AI-powered backend systems.

The experience strengthened:

- Problem-solving skills
- Software design thinking
- API development capabilities
- AI integration knowledge
- System architecture understanding

This program has prepared me to work on real-world backend and AI engineering projects with greater confidence and technical depth.